TASK 1 — CREATE PRODUCT API STRUCTURE

Aim

The objective of this task is to create the basic backend structure for a Product API using:

- Node.js
- Express.js
- MongoDB
- Mongoose

This backend application will be used to store and manage product details.

Step 1 — Create Project Folder

Create a new folder named:

```
product-api
```

Create the following folders:

- models
- routes
- controllers
- config

Create these files:

- server.js
- .env
- package.json

Initialize the Node.js project and install:

- express
- mongoose
- dotenv
- nodemon

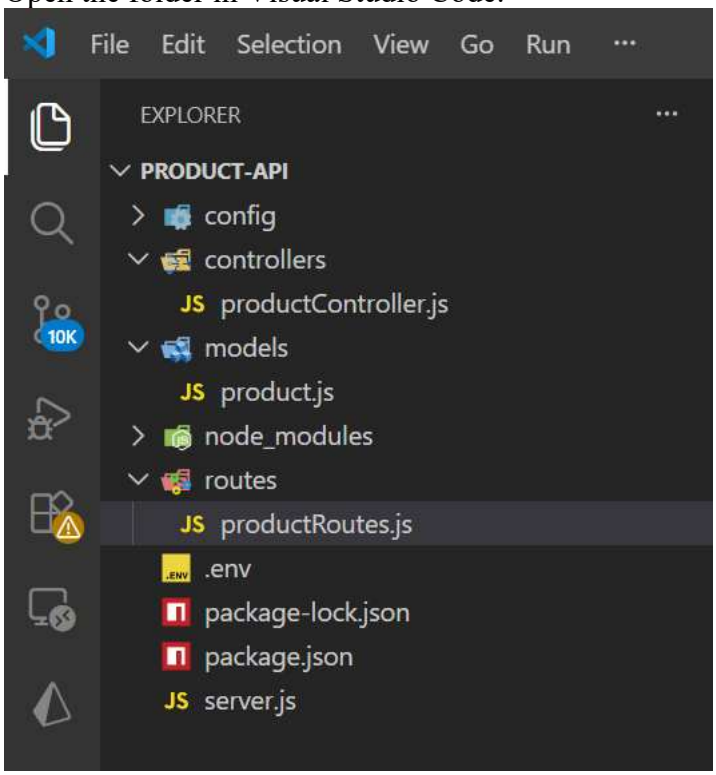
Setup:

D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

- Create a basic Express server
- Connect MongoDB

Open the folder in Visual Studio Code.



Step 2 — Initialize Node.js Project

Open terminal in VS Code and run:

```
npm init -y
```

This command creates a `package.json` file automatically.

The `package.json` file stores:

- project name
- dependencies
- scripts
- version details

Step 3 — Install Required Packages

D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

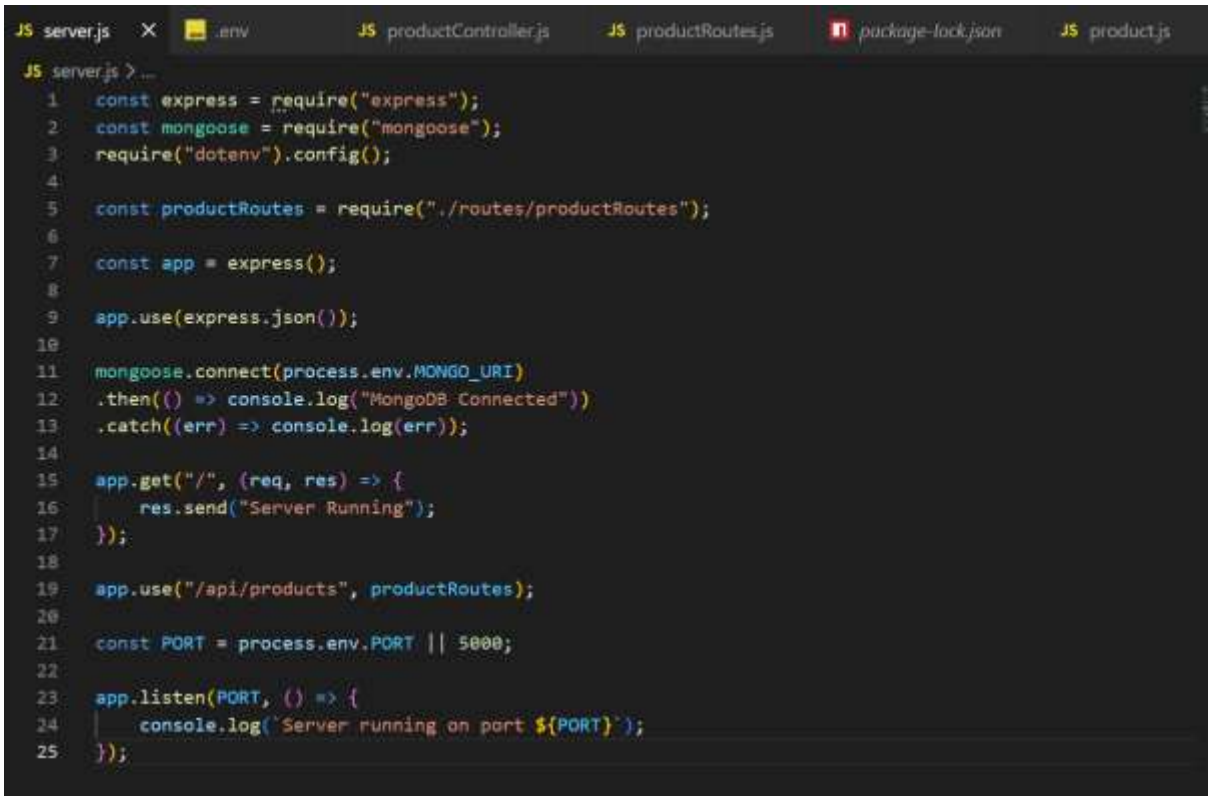
Install the required backend packages using:

```
npm install express mongoose dotenv
```

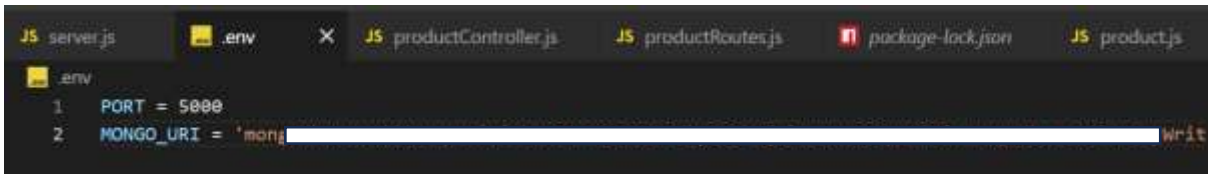
Install nodemon as development dependency:

```
npm install nodemon --save-dev
```

Server.js



.env



D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

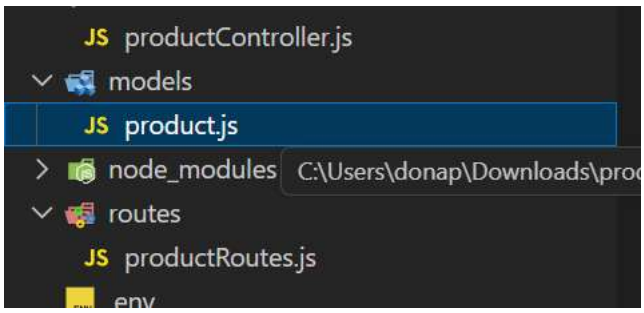
TASK 2 — Build Product Details API

Create a Product model with the following fields:

- productName
- price
- brand
- category
- stock

Create:

- Product model



Product.js

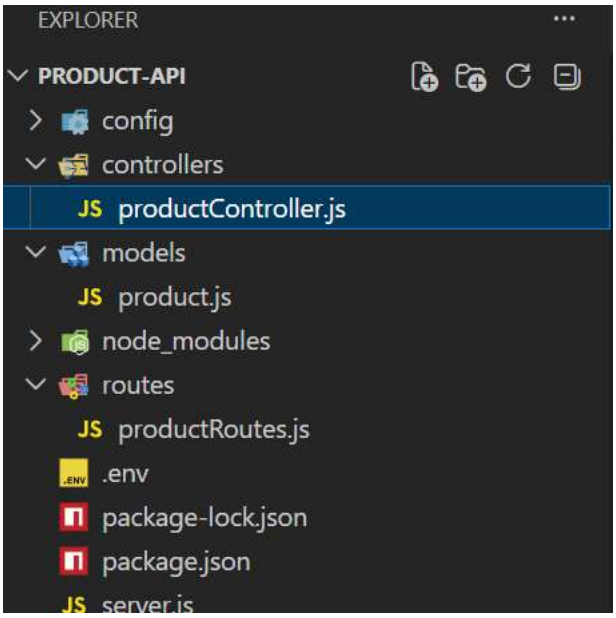
```
const productSchema = new mongoose.Schema({
    productName: {
        type: String,
        required: true
    },
    price: {
        type: Number,
        required: true
    },
    brand: {
        type: String,
        required: true
    },
    category: {
        type: String,
        required: true
    },
    stock: {
        type: Number,
        required: true
    }
});

module.exports = mongoose.model("Product", productSchema);
```

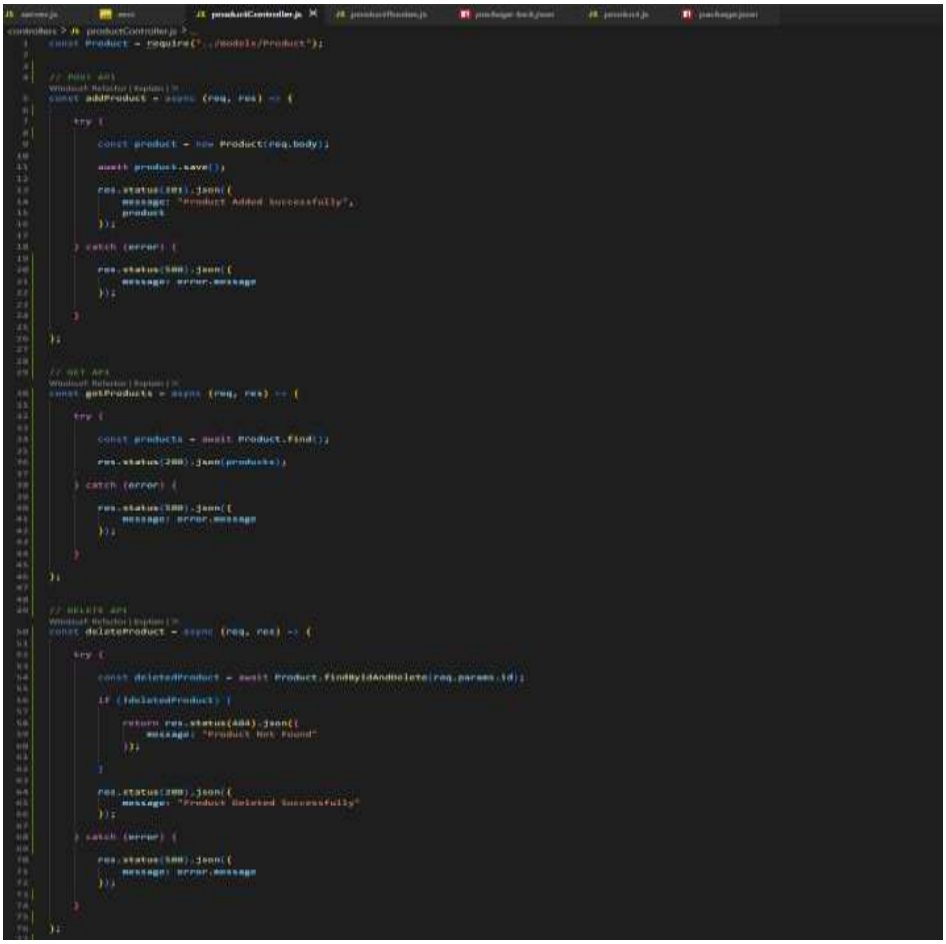
D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

- Product controller



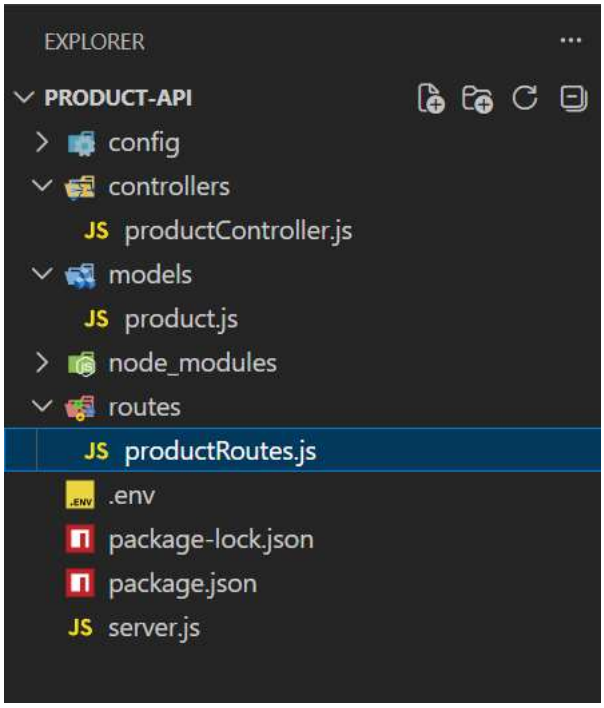
ProductController.js



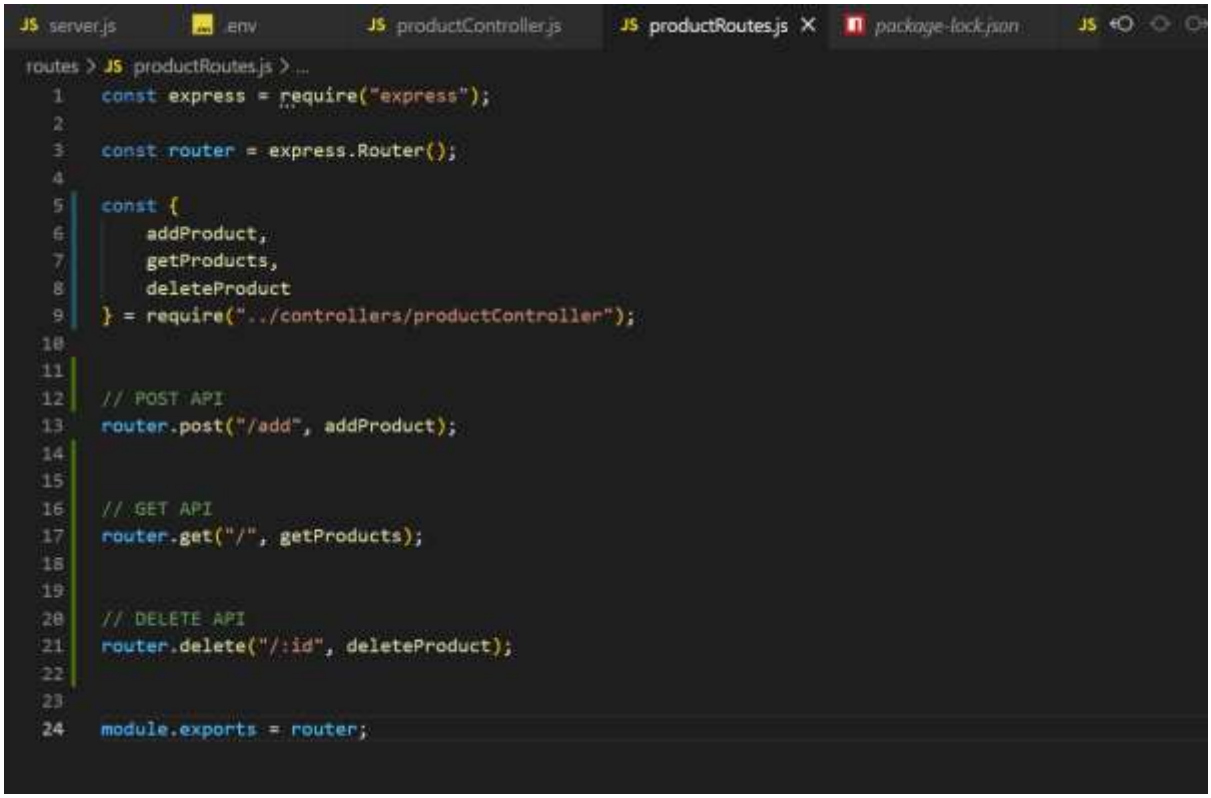
D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

- Product routes



productRoutes.js



D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

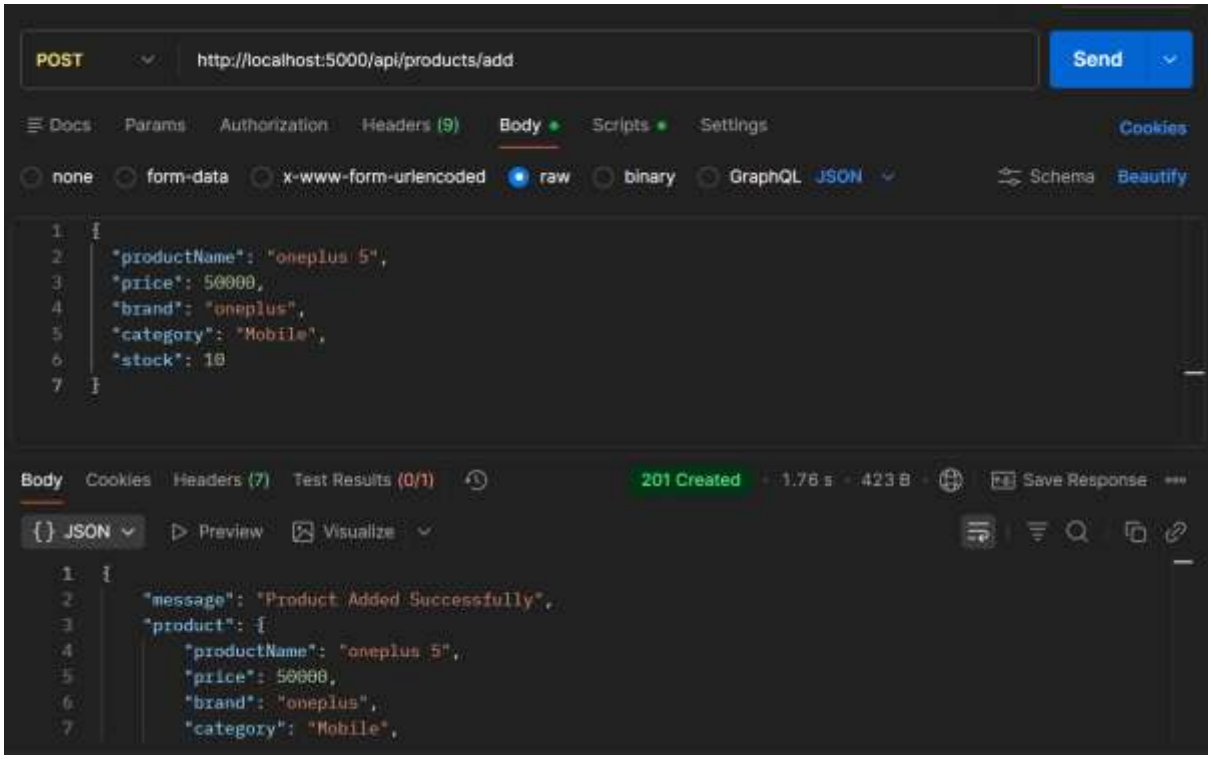
DAY 5 — BACKEND TASKS

Create POST API

API Endpoint:

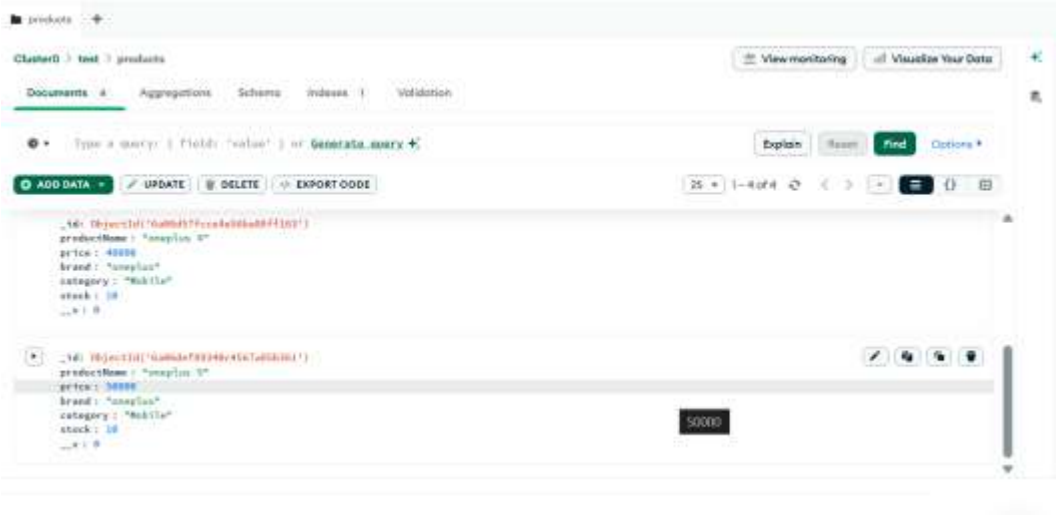
`/api/products/add`

In Postman



The API should:

- Accept product data
- Store data in MongoDB



- Return success message

D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com

DAY 5 — BACKEND TASKS

TASK 3 — Create Get Products API

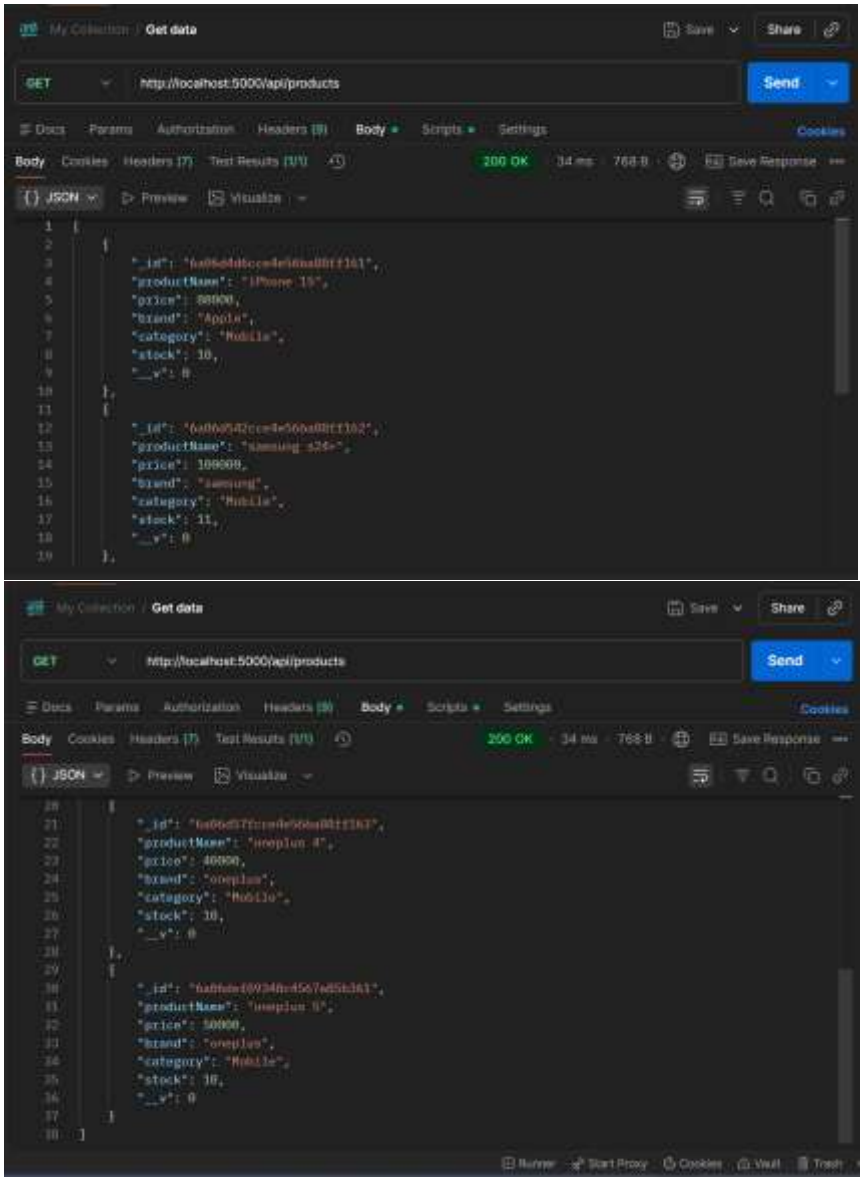
Create GET API

API Endpoint:

`/api/products`

The API should:

- Fetch all products from MongoDB
- Return product list
- **In postman**



D. Vishnu Vardhan Reddy
donapativishnu21@gmail.com